

DATA ENGINEERING FOUNDATIONS / SESSION 01

PySpark on **AWS**

From "my Node script ran out of memory" to a cluster chewing through 4 terabytes — and understanding every single box in the diagram.

AUTHOR

Surender Sara

COMPANY

NorthBay **Solutions**

ISSUED

August 2026

FORMAT

52 slides · 90 min

Lecture · 90 minutes · no laptops open

Hands-on labs follow separately

The deal

You are JavaScript developers. That is an advantage here, not a gap.

You already know this

- ▶ **Arrays.** map, filter, reduce, groupBy.
- ▶ **Promises.** Nothing runs until you await.
- ▶ **Workers.** Splitting work across threads.
- ▶ **JSON & APIs.** Data in, data out.

You don't know this — yet

- ▶ What happens when the array is **4 TB**.
- ▶ What happens when one machine **isn't enough**.
- ▶ What happens when machines **die mid-job**.
- ▶ Where AWS actually fits in any of it.

By minute 90 you will

- ▶ Read a Spark architecture diagram cold.
- ▶ Explain HDFS, shuffle, lazy evaluation.
- ▶ Pick EMR vs Glue vs Athena and defend it.
- ▶ Know the 5 ways your first job will fail.

Rule for the next 90 minutes: every single concept gets mapped back to something you already write in JavaScript. If a mapping doesn't land, stop me. You will not write code today — that is what the labs are for.

Ninety minutes, ten moves

- ▶ **01** The wall — why one machine stops working
- ▶ **02** Scale up vs scale out, and the one idea behind all of it
- ▶ **03** HDFS — a hard drive made of a hundred computers
- ▶ **04** MapReduce — the thing Spark replaced
- ▶ **05** Spark — the distributed array
- ▶ **06** PySpark — Python driving a JVM cluster
- ▶ **07** The AWS flip — S3 kills HDFS
- ▶ **08** Parquet, partitioning, Iceberg — where the 10x lives
- ▶ **09** One real job, end to end
- ▶ **10** Five ways your job dies, and the fixes

We build it bottom-up: storage → execution → language → cloud → failure. Every layer exists because the layer under it hit a wall.

01

The wall

The moment your laptop — and your instincts — stop working.

~8 MIN

A perfectly normal request

This exact scenario is the spine of the whole lecture. Keep it in your head.

The app you built

A React + Node e-commerce site. Every click, view, add-to-cart and checkout is logged as a JSON line and dropped into cloud storage.

It has been running for **18 months**. Nobody thought about it again.

What accumulated

4.2 TB

raw JSON logs

900 M

click events

2.1 M

individual files

Then product walks over.

"Give me the checkout funnel — view → cart → purchase — broken down by country and hour of day, for the whole 18 months. Need it Thursday."

16 GB of RAM meets 4.2 TB of data

YOUR FIRST INSTINCT

```
const fs = require('fs');

const raw = fs.readFileSync('events.json');
const rows = JSON.parse(raw);

const funnel = rows
  .filter(r => r.type === 'purchase')
  .reduce(groupByCountryHour, {});
```

WHAT NODE SAYS

<--- Last few GCs --->

FATAL ERROR: Reached heap limit
Allocation failed – JavaScript heap
out of memory

4,200,000 MB of data
16,000 MB of RAM
262x too big.

Note what actually broke. Not your logic — the funnel code is correct. The *assumption* broke: that the data fits in one process's memory. Every tool in this lecture exists to kill that assumption.

"I'll just stream it"

Better. Still doomed. Three separate walls, and streaming only dents one.

Wall 1 — Memory

Streaming fixes reading. It does **not** fix **groupBy**.

Your accumulator holds every country × hour × user in RAM. The intermediate state is the problem, not the input.

Wall 2 — Time

One NVMe SSD reads ~**2 GB/s** on a good day. Network-attached storage: ~**250 MB/s**.

$4.2 \text{ TB} \div 250 \text{ MB/s} = \text{4.7 hours}$ just to move the bytes. Before you compute anything.

Wall 3 — Failure

Hour 4 of 5. The process crashes. A spot instance is reclaimed. The kernel OOM-kills you.

You start again from **zero**. There is no checkpoint, no retry, no partial credit.

These three walls are the entire field.

Distributed data engineering is not a fancier way to write map/filter/reduce. It is the discipline of making map/filter/reduce survive memory limits, I/O limits, and hardware that dies.

02

Scale up or scale out

Two doors. One of them is a dead end that costs a fortune.

~6 MIN

Two doors

Every big-data tool ever built walked through the second one.

Door 1 — Scale UP (vertical)

Buy a bigger machine. AWS will sell you an **x1e.32xlarge**: 128 vCPU, **3,904 GB RAM**.

~**\$26.69/hour**. \$19,000/month, running or idle.

And your 4.2 TB still doesn't fit. There is no bigger box. You have hit a **physical ceiling**, and price scales super-linearly on the way to it.

\$26.69

1 huge machine · 128 vCPU

\$15.36

40 small machines · 320 vCPU

Door 2 — Scale OUT (horizontal)

Buy 40 boring machines. **m5.2xlarge**: 8 vCPU, 32 GB RAM, **\$0.384/hour** each.

~**\$15.36/hour** for 320 vCPU and 1,280 GB RAM.

Half the price, 2.5× the cores. Add more whenever you want. Price scales **linearly, forever**.

2.5x

more compute

∞

headroom — just add nodes

Move the code to the data

The naive move

Data lives in storage. Compute lives on your machine. So: pull the data to the compute.

4.2 TB across the network. The network is now your wall, and it is the slowest thing in the building.

The move that changed everything

Your funnel code is **50 kilobytes**. The data is **4.2 terabytes**.

So don't move the data. **Ship the 50 KB to wherever each block of data already lives** and run it there.

Move the data

4.2 TB → network → 1 machine
hours, and one bottleneck

VS

Move the code

50 KB → network → 40 machines
milliseconds, 40 parallel readers

Data locality. Remember this phrase. It is the single idea underneath HDFS, MapReduce, Spark, and every design decision they made. Everything else is plumbing.

Source: <https://www.youtube.com/watch?v=Uw13p4Rm2W8>, NorthBay Solutions

03

HDFS

One hard drive, made out of a hundred computers that are all failing.

~10 MIN

Where all of this came from

2003-2004 — Google publishes

Google had the same problem at planet scale and published two papers instead of products: **GFS** (a distributed filesystem) and **MapReduce** (a distributed compute model).

2006 — Hadoop

Doug Cutting and Mike Cafarella build open-source implementations at Yahoo. Named after his son's toy elephant. That is the whole story of the logo.

Today

Almost nobody runs on-prem HDFS anymore. **You still must understand it**, because Spark's execution model, its vocabulary, and half of AWS's data services are shaped by it.

Hadoop was two things. **HDFS** = how to store a file across many machines. **MapReduce** = how to compute across many machines. Spark replaced the second one. The first one got replaced by S3.

A file is not a file

A 1 GB FILE, THE WAY HDFS SEES IT



Split

Fixed **128 MB** blocks. A file is no longer a file — it is a *list of block IDs*.

Scatter

Blocks land on different machines, so 8 disks read your file **at the same time**.

Replicate

Every block exists **3 times**, on 3 machines, across ≥ 2 racks. Disks die daily.

The NameNode is a Map

If you understand JavaScript's Map, you already understand the NameNode.

NAMENODE — THE INDEX

```
// Mentally, it is this:
const namenode = new Map([
  ['/logs/events.json', {
    size: 1073741824,
    blocks: [
      { id: 'blk_001',
        at: ['nodeA', 'nodeB', 'nodeF'] },
      { id: 'blk_002',
        at: ['nodeB', 'nodeC', 'nodeG'] },
    ]
  }
]);
```

- ▶ The NameNode stores **metadata only** — names, permissions, block IDs, which machines hold them. Never your actual bytes.
- ▶ It keeps all of that **in RAM** for speed. That is why it has a hard limit on *number of files*, not total size.
- ▶ **Read path:** client asks NameNode → gets block locations → then talks *directly* to DataNodes. The NameNode is never in the data path.
- ▶ It was Hadoop's original single point of failure. Fixed later with a standby NameNode + shared edit log (**HA**).

Three numbers that explain everything

Why 3 copies?

At 1,000 disks, several fail **every week**. Failure is not an exception to handle — it is the steady state.

3 replicas: 1 on the writing node, 1 on another node in the same rack, 1 in a **different rack**. Survives a disk, a machine, and a whole rack's switch.

Why 128 MB blocks?

A disk seek costs ~**10 ms**. Transfer runs at ~100 MB/s. Make blocks big enough that seek time becomes noise — ~1% of transfer time.

Too small → seek-bound and NameNode RAM explodes. Too big → you lose parallelism and retries get expensive.

The trap this creates

Each file costs ~**150 bytes** of NameNode RAM regardless of size.

Your 2.1 million tiny JSON files? They cost **more metadata and more tasks** than 2.1 million would in one big file. This is **the small files problem** and it will bite you on AWS too.

The bill comes due

WHAT HDFS BOUGHT

- ▶ One logical filesystem over hundreds of machines
- ▶ Dozens of disks reading **in parallel**
- ▶ Automatic replication & re-replication on failure
- ▶ **Data locality** — compute scheduled where blocks live
- ▶ Cheap commodity hardware instead of a SAN

WHAT HDFS COST

- ▶ Storage and compute **welded together** — need more disk, buy more CPU
- ▶ The cluster runs **24/7**, even when idle, because it *is* your storage
- ▶ 3× replication = you buy 3 TB of disk per 1 TB of data
- ▶ You are now a full-time cluster administrator
- ▶ Immutable, append-only. No updates, no deletes.

Hold this thought. Every one of those five costs is precisely what S3 fixes in Section 07. HDFS is the *why*; S3 is the *answer*. You cannot appreciate the answer without the why.

04

MapReduce

The compute model Spark exists to replace — and why it deserved to die.

~ 7 MIN

MapReduce is **map** and **reduce**. Really.

The shuffle in the middle is the only genuinely new idea. Everything else you know.

YOU HAVE WRITTEN THIS A THOUSAND TIMES

```
const counts = lines
  .flatMap(l => l.split(' ')) // MAP
  .reduce((acc, w) => {       // REDUCE
    acc[w] = (acc[w] || 0) + 1;
    return acc;
  }, {});
```

One machine. One thread. One line of thought.

MAPREDUCE IS THE SAME THREE MOVES

MAP

each machine, on its own local block:

emit (word, 1) per word

runs in parallel, no coordination



SHUFFLE

route every pair across the network so

all ("the", 1) land on the same machine

all-to-all. the expensive part.



REDUCE

each machine sums the group it received

→ ("the", 84213)

parallel again

Three fatal flaws

Flaw 1 — Disk between every step

Map output is written to disk. Shuffled over the network. Read from disk. Reduced. Written to HDFS — **with 3× replication**.

Then the next job reads it all back off disk again.

Flaw 2 — Only two verbs

A real funnel query is 6-8 logical steps. MapReduce gives you two. So you chain **six separate MapReduce jobs**, each with a full disk round-trip between them.

Flaw 3 — Iteration is fatal

Machine learning re-reads the same dataset 100 times. MapReduce re-reads it **from disk** 100 times.

The data never gets to stay in memory. Not once.

WHAT THAT FEELS LIKE IN JS TERMS

```
// MapReduce, honestly:
const a = arr.map(f);
await fs.writeFile('/tmp/1', a);  ← to disk. 3x.
const b = JSON.parse(await fs.readFile('/tmp/1'));
const c = b.filter(g);
await fs.writeFile('/tmp/2', c);  ← again
// ...repeat for every single step
```

2009, Berkeley AMPLab.

Matei Zaharia asks the obvious question: **what if we just kept it in memory between the steps?**

That is Spark. Same programming model, **10-100× faster**, and about **1/20th the code**.

05

Spark

A distributed array with a lazy, self-optimising query planner bolted on.

~16 MIN

The Rosetta Stone

Photograph this slide. Every confusing Spark error traces back to a row in this table.

Spark in one sentence.

A **distributed array** you manipulate with map/filter/groupBy, that **doesn't execute anything** until you ask for a result, and that **rewrites your code** into a faster plan before it runs.

WHAT YOU KNOW IN JAVASCRIPT	WHAT IT IS CALLED IN SPARK	THE DIFFERENCE THAT MATTERS
<code>Array<T></code> in one process	DataFrame — rows split into partitions	Lives on 40 machines at once
<code>.map()</code> / <code>.filter()</code>	Transformations — <code>select, filter, withColumn</code>	Lazy. Nothing runs. It just records intent.
<code>.reduce()</code> / grouping	Wide transformations — <code>groupBy().agg()</code> , <code>join</code>	Triggers a network shuffle
<code>new Promise(...)</code> — not awaited	A chain of transformations	Both are a description of future work
<code>await</code> / <code>.then()</code>	Actions — <code>show, count, collect, write</code>	<i>This</i> is what makes the cluster move
<code>worker_threads</code> on your box	Executors on other machines	Separate JVMs, separate hosts, can die

Partitions are the unit of everything

- ▶ A DataFrame is chopped into **partitions**. A partition is just a chunk of rows that lives in one executor's memory.
- ▶ **1 partition = 1 task = 1 CPU core**. That is the entire parallelism story.
- ▶ Your 4.2 TB at 128 MB per partition → roughly **33,000 partitions**.
- ▶ With 320 cores, Spark runs 320 tasks at a time and works through the queue in **~103 waves**.
- ▶ Too few partitions → idle cores, and one task must hold too much in memory. Too many → scheduling overhead swamps the actual work.

Rule of thumb

Target **2-4× your total core count** in partitions, and **~128-200 MB** per partition.

```
df.rdd.getNumPartitions()
```

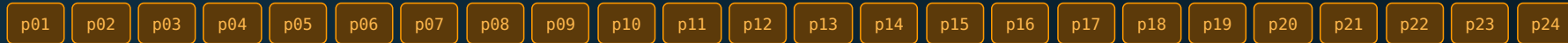
```
df.repartition(320)
```

 — full shuffle, exact count

```
df.coalesce(40)
```

 — merges only, no shuffle, cheaper

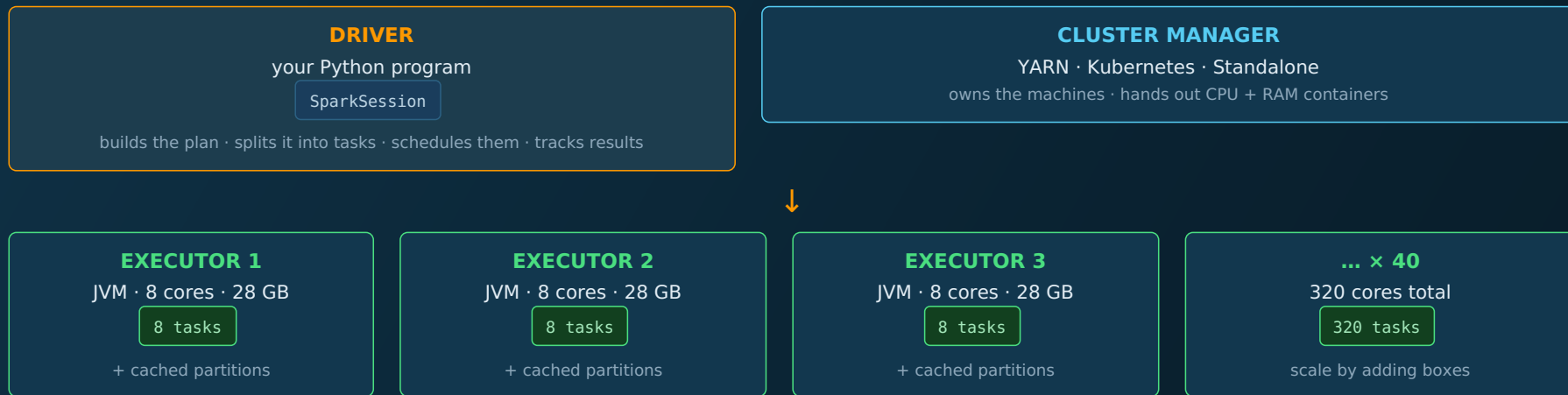
Partition count is the #1 knob you will touch in the labs.



... × 33,000

Each block is one task, one core, ~128 MB of your 4.2 TB. Spark's job is to schedule these onto free cores as fast as it can.

Three boxes. That's the whole cluster.



SPARK	THE NODE.JS EQUIVALENT	WHAT IT ACTUALLY DOES
Driver	your <code>index.js</code> main process	Builds the DAG, schedules tasks, collects results. One per application.
Cluster Manager	the OS scheduler, for a fleet	Hands out containers of CPU/RAM. On EMR this is YARN .
Executor	a <code>worker_thread</code> , on another machine	Long-lived JVM. Runs tasks, holds cache. Can die without killing the job.
Task	one <code>.map()</code> callback call	The work for exactly one partition, on exactly one core.

Where beginners kill their cluster

The Driver is a single point of failure

It holds the plan, the schedule, and the results you pull back. **Driver dies → entire application dies.**

Executors dying is *fine* — Spark just recomputes their lost partitions elsewhere. That is the whole point of the lineage graph.

Which is why this line is a crime

```
# 900 million rows → one Python process  
rows = df.collect()
```

```
df.show(20)
```

```
df.limit(1000).toPandas()
```

```
df.write.parquet(...)
```

Bring back **summaries**, never the dataset. If it has to fit in the driver, you didn't need Spark.

Executor sizing, since someone always asks: fat executors (32 cores) waste memory on GC pauses; skinny ones (1 core) lose all broadcast and cache sharing. **4-5 cores and 20-30 GB per executor** is the boring answer that is almost always right.

Nothing runs until you **await**

A chain of transformations is an unawaited Promise. The action is the await.

TRANSFORMATIONS — LAZY, RETURN A NEW DATAFRAME

select filter / where withColumn drop groupBy join

orderBy distinct union repartition

ACTIONS — EAGER, RETURN DATA OR WRITE FILES

show count collect first / take write toPandas

foreach

If you can't remember which is which: does it return another DataFrame? Then it's lazy.

```
df = spark.read.json("s3://.../events/") # lazy
df = df.filter(col("type")=="purchase")  # lazy
df = df.withColumn("hr", hour("ts"))      # lazy
df = df.groupBy("country", "hr").count()  # lazy
```

```
# Nothing has happened. No cluster activity.
# You have built a recipe, not a meal.
```

```
df.show() # ⚡ NOW 320 cores wake up
```

Laziness buys you a compiler

Why laziness is a superpower, not a quirk

Because Spark sees your **whole chain** before running a single row, it can rewrite it. An eager system executes step 1 before it knows step 4 exists.

Predicate pushdown: your filter gets shoved down into the file reader, so the columns and row-groups you don't need are *never read off disk*.

Column pruning: you selected 3 of 40 columns → Spark reads 3.

4.2 TB

what you wrote:
read everything

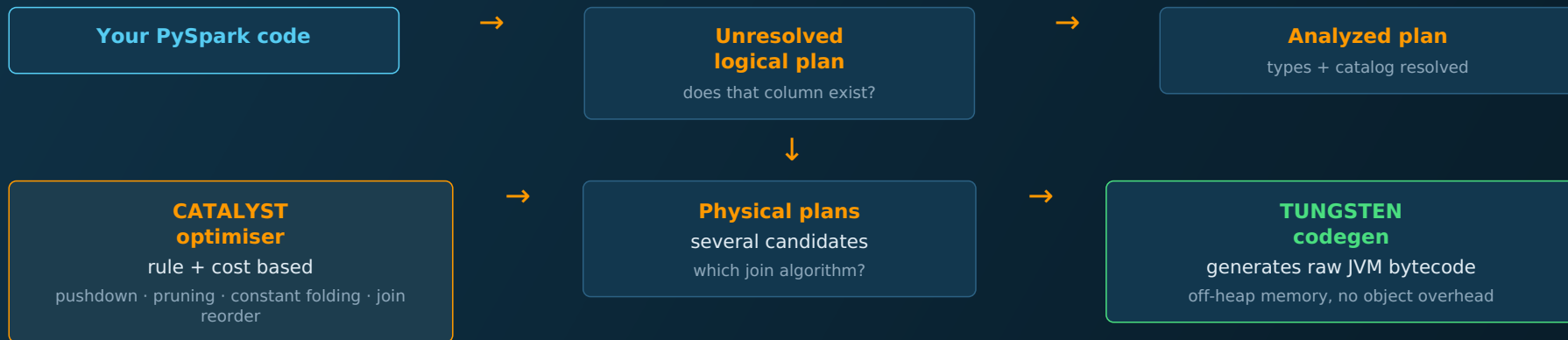
40 GB

what actually gets read
after optimisation

```
# See it yourself – you will do this in lab 3
df.explain(True)

== Physical Plan ==
*(2) HashAggregate(keys=[country, hr])
+- Exchange hashpartitioning(country, hr, 200)
   +- FileScan parquet [country,hr,user_id]
      PushedFilters: [EqualTo(type,purchase)]
      PartitionFilters: [year=2026]
```

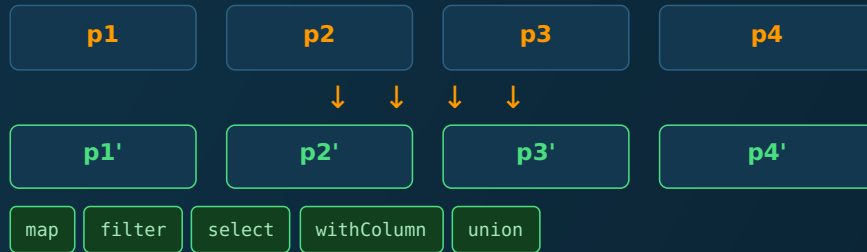
What happens between `.show()` and the answer



This is why Python is not slow here. Your PySpark code is not processing rows. It is *emitting a query plan*. Catalyst optimises it and Tungsten compiles it to JVM bytecode that runs on off-heap memory. Python never touches the data.

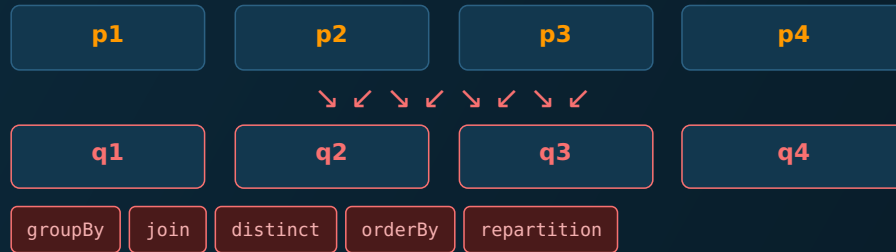
Narrow vs wide — the only distinction that matters for speed

NARROW — CHEAP, PARALLEL, NO NETWORK



Each output partition depends on exactly one input partition. Runs entirely in place, in memory, no coordination.

WIDE — EXPENSIVE, NETWORK + DISK



Every output partition may need rows from every input partition. This is a SHUFFLE.

The single most expensive thing you can do

- ▶ A shuffle is an **all-to-all data movement**: every executor writes files, every other executor reads from them across the network.
- ▶ $200 \text{ source partitions} \times 200 \text{ target partitions} = \mathbf{40,000 \text{ shuffle blocks}}$ written to local disk and fetched over the wire.
- ▶ It is the only place Spark **must** hit disk. It serialises, spills, and re-reads.
- ▶ A shuffle **ends a stage**. Spark can't start the next stage until the last task of this one finishes — so one slow task stalls 319 idle cores.
- ▶ **90% of "my Spark job is slow" is a shuffle problem.**

The hierarchy, precisely

Application — one SparkSession, one driver.

Job — one action (`show` , `write`) triggers one job.

Stage — a run of narrow transformations. **Stage boundaries are shuffles.**

Task — one partition, one core, one stage.

You will see exactly this tree in the Spark UI in lab 4. Count the stages = count the shuffles.

Everything you will ever tune — broadcast joins, partition counts, salting, bucketing, AQE, filter placement — is you trying to **avoid a shuffle, shrink a shuffle, or balance a shuffle**. That is the job.

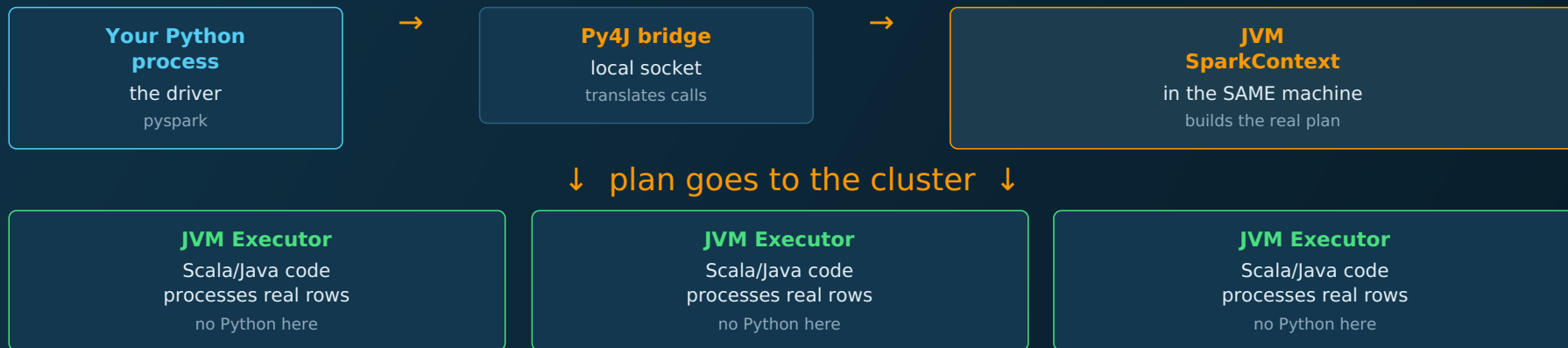
06

PySpark

Python holding the steering wheel of a JVM cluster.

~8 MIN

Python drives. The JVM does the work.



The fact that surprises everyone.

For the **DataFrame API**, PySpark is **exactly as fast as Scala Spark**. Your Python builds a plan; the JVM executes it. Python is a remote control, not a bottleneck.

The exception: UDFs

WHAT BREAKS THAT — THE PYTHON UDF

```
# Every single row: JVM → pickle → Python
# process → pickle → back to JVM.
# 900,000,000 times.
```

```
@udf("string")
def clean(s):
    return s.strip().lower()
```

```
df.withColumn("c", clean("country"))
# → 10–100x slower. Catalyst can't
# see inside it, so it can't optimise it.
```

WHAT TO DO INSTEAD

```
# 1. Built-in functions — ALWAYS look first.
#    ~300 of them. Compiled to JVM bytecode.
from pyspark.sql.functions import lower, trim
df.withColumn("c", lower(trim("country")))
```

```
# 2. If you truly must use Python:
#    pandas UDF — Apache Arrow moves a whole
#    batch at once. ~10x faster than plain UDF.
@pandas_udf("string")
def clean(s: pd.Series) -> pd.Series:
    return s.str.strip().str.lower()
```

The rule you will repeat to juniors forever: a built-in function beats a pandas UDF beats a Python UDF. Reach down the list only when the one above genuinely can't do it.

Your product question, answered

4.2 TB. 900 million rows. This is the entire program.

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, hour, countDistinct

spark = SparkSession.builder.appName("funnel").getOrCreate()

events = (spark.read
    .parquet("s3://shop-lake/curated/events/")      # lazy
    .filter(col("year") >= 2025)                    # partition pruning
    .filter(col("event_type").isin("view", "cart", "purchase"))
    .withColumn("hr", hour("event_ts")))

funnel = (events
    .groupBy("country", "hr", "event_type")        # ← SHUFFLE
    .agg(countDistinct("user_id").alias("users")))

(funnel.write
    .mode("overwrite")
    .partitionBy("country")
    .parquet("s3://shop-lake/marts/funnel/"))      # ← ACTION. Cluster runs.
```

Count the lines. Sixteen. In MapReduce this is ~400 lines of Java across six chained jobs.

Count the actions. One — the write. Everything above it is a plan Catalyst may rewrite.

07

The AWS flip

S3 quietly killed HDFS, and that one change reshaped everything above it.

~14 MIN

S3 replaces HDFS — and everything changes

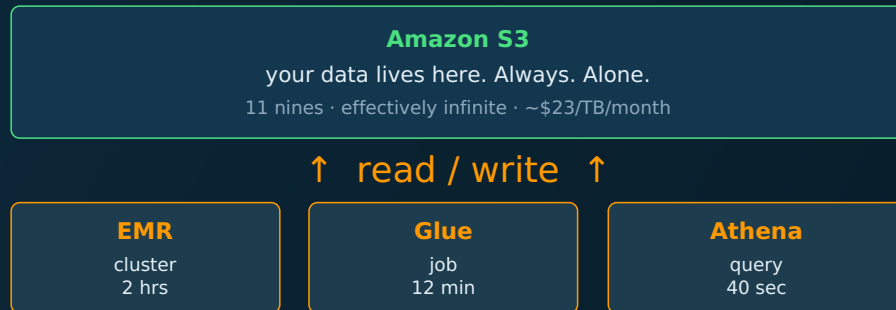
This is the single biggest architectural difference between Hadoop and modern AWS data engineering.

THE OLD WORLD — COUPLED



Same box holds the CPU and the disk. Need 1 more TB? Buy a whole server. Job done at 3 a.m.? It still runs — switching it off **deletes your data**.

THE NEW WORLD — DECOUPLED



Compute is **rented by the second**, then thrown away. Three engines can hit the *same* data at once, and nothing is lost when they stop.

What you gain, what you pay

What you gain

- ▶ Storage and compute **scale independently**
- ▶ **Zero** cost when no job is running
- ▶ 11 nines durability, no replication to manage
- ▶ Many engines, one copy of the data
- ▶ Clusters become **disposable** — spin up, run, kill

What you give up

- ▶ **No data locality.** Everything crosses the network — but AWS networking is now fast enough that this lost
- ▶ S3 is an **object store**, not a filesystem: no real directories, **no atomic rename**
- ▶ Listing millions of objects is slow — the small-files tax gets worse
- ▶ Per-request costs (GET/PUT/LIST) become real at scale

Why it won anyway

The idle-cluster bill was murdering everyone. A Hadoop cluster sized for the 2 a.m. peak sits at **~20% utilisation** the rest of the day — and you pay 100% of it, 24/7, forever.

S3 + ephemeral compute turned a **capital expense you babysit** into **a line item you only pay while it runs**.

The one gotcha to actually remember: Spark's default output committer writes to a temp path then renames. On a real filesystem rename is instant. On S3 it is a *full copy*. Use the **S3A / EMRFS optimised committers** — on EMR and Glue they are on by default, but the moment you run Spark on plain EC2 or EKS, this is your problem.

If S3 is the storage, what is the NameNode?

Glue Data Catalog

THE NAMENODE'S REPLACEMENT — FOR SCHEMA

S3 stores bytes. It has no idea your files are a table with columns and types. The **catalog** holds that: databases, tables, columns, types, partitions, where the files are.

It is a managed **Hive metastore**. Athena, EMR, Glue, Redshift Spectrum and SageMaker all read the same one.

Crawlers — and a warning

A crawler scans an S3 prefix, infers schema, and registers/updates the table.

Convenient in a demo, a liability in production. Inference drifts, types flip, and one bad file changes your schema silently. In real pipelines you **declare** the schema in code or in Iceberg.

The mental swap

NameNode knew where blocks were.

S3 knows where objects are.

Glue Catalog knows what those objects *mean*.

Two jobs the NameNode used to do, now split cleanly in two.

Six ways to run Spark on AWS

They all execute the same Apache Spark. What differs is who manages the machines and how you get billed.

SERVICE	WHAT IT IS	PRICING SHAPE	REACH FOR IT WHEN
EMR on EC2	Managed Hadoop/Spark cluster on EC2 you can SSH into. YARN, full control, custom AMIs, bootstrap actions.	EC2 + ~25% EMR fee. Spot on task nodes = up to 90% off.	Long or heavy jobs, custom native libs, you want max control and lowest cost per TB.
EMR Serverless	Same Spark runtime, no cluster to size or manage. You submit a job; capacity appears.	Per vCPU-hour + GB-hour, billed while running.	The sane default for most teams. Spiky or unpredictable workloads.
AWS Glue (Spark)	Serverless Spark plus catalog, crawlers, bookmarks, visual editor. Glue 5.1 → Spark 3.5.6, Python 3.11.	Per DPU-hour (1 DPU = 4 vCPU + 16 GB).	ETL that is tightly coupled to the catalog; teams that want the least ops surface.
Athena	Serverless SQL over S3 (Trino), plus a Spark flavour for notebooks.	~\$5 per TB scanned. Nothing when idle.	Ad-hoc SQL, exploration, BI. Not for heavy transformation pipelines.
EMR on EKS	Spark as pods on your existing Kubernetes cluster.	EKS + EC2 underneath.	You already run everything on K8s and want one control plane.
Redshift / Spectrum	MPP data warehouse; Spectrum queries S3 from inside it.	Provisioned nodes or Serverless RPU.	Serving layer for BI with concurrency and sub-second SQL — not for ETL.

So which one do I actually pick?

Default answer

Start on **EMR Serverless** or **Glue**. No cluster sizing, no idle bill, no ops. Move to EMR on EC2 *only* when you can point at a number that proves it is cheaper.

Cost order, roughly

Cheapest EMR on EC2 with spot task nodes

EMR Serverless
Glue

Priciest per TB Athena, if you scan carelessly

...and every one of those flips if your job is badly written.

EMR cluster anatomy

Primary — runs YARN ResourceManager + the driver. Never on spot.

Core — run tasks and hold HDFS/shuffle data. On-demand.

Task — compute only, no HDFS. **Put these on spot.** They can vanish and Spark just retries.

~90%

spot discount on task nodes

~\$5

per TB scanned, Athena

~\$23

per TB / month, S3 Standard

\$0

compute cost while nothing runs

Illustrative us-east-1 list prices — they move, and they differ by region. Always price your own workload with the AWS Pricing Calculator before you commit to an architecture.

08

Parquet & friends

The unglamorous slide where the real 10x is hiding.

~6 MIN

JSON is a transport format, not a storage format

ROW FORMAT — JSON, CSV

u1	US	14	buy	49.90	...35 more
u2	DE	09	view	0.00	...35 more
u3	IN	22	cart	12.50	...35 more

To read **one column** you must read **every byte of every row**. No compression worth the name — the values next to each other are unrelated.

COLUMNAR — PARQUET, ORC

u1 u2 u3 u4 ...	user_id
US DE IN US ...	country → dictionary-encodes to near nothing
14 09 22 03 ...	hour → run-length + bit-packing

Read **only the columns you asked for**. Identical values sit together, so compression is brutal. Each row-group carries **min/max stats**, so Spark skips whole chunks without decoding them.

4.2 TB

raw JSON

380 GB

Parquet + Snappy

12 GB

actually read for a 3-column query

~350x

less I/O than the naive path

Partitioning: folders that act like an index

PHYSICAL LAYOUT ON S3

```
s3://shop-lake/curated/events/  
  year=2026/  
    month=08/  
      day=30/  
        part-00000.snappy.parquet 128 MB  
        part-00001.snappy.parquet 128 MB  
      day=31/  
        part-00000.snappy.parquet
```

```
# Spark reads the folder names as columns  
df.filter("year=2026 AND month=8")  
# → never even LISTS the other folders
```

- ▶ **Partition on what you filter on** — usually date. Then country or tenant.
- ▶ **Never** partition on something high-cardinality like `user_id`. 900 million directories will end you.
- ▶ Aim for **≥128 MB per partition folder**. Under ~10 MB and you have recreated the small-files problem in the cloud, where listing costs money.
- ▶ Forget the partition filter and you scan the whole table. On Athena that is a **bill**, not just a wait.

Parquet can't do transactions. Iceberg can.

The problem Parquet still has

Parquet files are **immutable**. There is no UPDATE, no DELETE, no transaction. Two writers at once corrupt each other. A reader can catch a half-written commit.

And GDPR says "delete this user" — across 380 GB of immutable files.

Iceberg / Delta / Hudi

A **metadata layer** on top of Parquet that adds a transaction log. The files stay Parquet; the log makes them a real table.

ACID commits · **MERGE / UPDATE / DELETE** · **time travel** to any past snapshot · safe schema evolution · hidden partitioning.

On AWS, in 2026

Apache Iceberg is the one to learn. It is first-class in Glue, EMR, Athena and Redshift, and the Glue Catalog speaks it natively.

This is what people mean by "**lakehouse**" — data lake economics with warehouse guarantees.

```
-- things a plain Parquet lake simply cannot do
MERGE INTO events t USING updates s ON t.id = s.id
  WHEN MATCHED THEN UPDATE SET * WHEN NOT MATCHED THEN INSERT *;

DELETE FROM events WHERE user_id = 'u-8842';           -- GDPR, in one line
SELECT * FROM events FOR TIMESTAMP AS OF '2026-08-01'; -- time travel
```

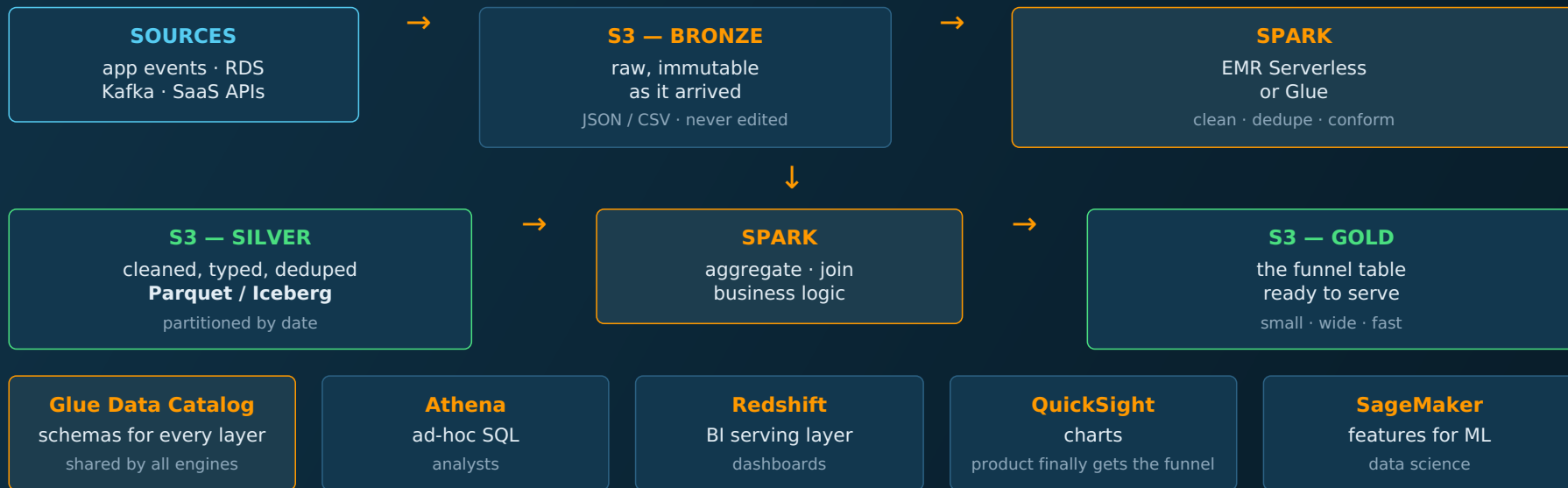
09

The whole picture

Every box you have met, in one architecture.

~ 7 MIN

What a real AWS data platform looks like



Bronze / Silver / Gold (also called the medallion architecture). Raw is never deleted, so any bug in Silver or Gold can be fixed by **reprocessing**, not by apologising to product about lost data. Orchestrate it with Step Functions, MWAA (Airflow), or Glue Workflows.

Reading the Spark UI

Where to look, in order

- ▶ **Jobs tab** — one row per action. Which one is slow?
- ▶ **Stages tab** — count them. Each boundary is a shuffle.
- ▶ **Event timeline** — are the bars solid, or is everything waiting on one task?
- ▶ **Task summary** — compare **min** / **median** / **max** duration. Max >> median means **skew**.
- ▶ **Shuffle read / write** — hundreds of GB here is your bill and your latency.
- ▶ **SQL tab** — the physical plan with real row counts. The truth lives here.

The six questions to ask a slow job

- ▶ Am I reading more data than I need? (partition filter, column pruning)
- ▶ How many shuffles? Can any be removed or broadcast away?
- ▶ Is one task doing 80% of the work? (skew)
- ▶ How many partitions vs how many cores?
- ▶ Am I calling a Python UDF where a built-in exists?
- ▶ Millions of tiny files on the read side?

Learn the Spark UI early. The engineers who get good at this fast are the ones who read the UI instead of guessing and re-running. Guessing costs you an hour per attempt; the UI costs you ninety seconds. Lab 4 is nothing but this.

10

How it dies

Five failures. You will hit at least three of them in the labs.

~8 MIN

Three of five

1 · Driver OOM

THE BEGINNER CLASSIC

```
df.collect()
df.toPandas()
```

900 M rows into one Python process.
Instant death.

Fix: `show(20)`, `limit().toPandas()`, or
write to S3 and query it.

2 · Data skew

THE ONE THAT WASTES YOUR
AFTERNOON

60% of your events are `country='US'`.

After the groupBy, **one partition holds 540 M rows.**

199 tasks finish in 20 s. One runs 40 minutes. 319 cores sit idle watching it.

Fix: enable **AQE** (`adaptive.enabled`), on
by default in Spark 3.x), or **salt** the key:
append a random 0-9 suffix, aggregate,
then re-aggregate.

3 · The exploding join

THE EXPENSIVE ONE

Joining 900 M rows to a 4 MB country lookup
triggers a **full shuffle of both sides**
because Spark didn't know the small side
was small.

```
from ...functions import broadcast
big.join(broadcast(small), "cc")
```

Ships the 4 MB to every executor. **Shuffle gone.** Often a 10x win in one line.

Two of five, plus one you'll do to yourself

4 · The small files problem

IT FOLLOWED YOU TO THE CLOUD

2.1 M files → 2.1 M tasks, each ~2 MB.
Scheduling overhead now exceeds the actual work, and S3 **LIST + GET charges** stack up on top.

Fix: a compaction job. Read the mess, `coalesce()`, write ~128 MB Parquet files.

Iceberg has `rewrite_data_files` built in.

Then guard the write side so it never happens again.

5 · The full-table scan

THE ONE WITH A DOLLAR SIGN

You forgot the partition filter. Spark reads all 380 GB instead of 12. On Athena that is a real invoice line.

Fix: always filter on the partition column, always `.explain()` and check

`PartitionFilters` is populated before you run at scale.

Bonus · caching wrong

THE CARGO CULT

`.cache()` is not free. It costs executor memory and evicts other things.

Cache when you reuse a DataFrame **3+ times**. Otherwise you have made the job slower and felt clever doing it. Call

`.unpersist()` when done.

Notice the pattern. Four of these five are about **moving data you didn't need to move**. Spark performance is almost never about CPU. It is about I/O and the shuffle.

Six things. If you remember nothing else.

1 · The one idea

Move the **code** to the data. Never the data to the code. Everything else is an implementation detail of that sentence.

2 · Lazy is a compiler

Transformations build a plan. Actions run it. Catalyst rewrites the plan first — which is only possible because it is lazy.

3 · Partitions are parallelism

1 partition = 1 task = 1 core. Every tuning conversation you will ever have starts here.

4 · The shuffle is the enemy

Narrow is cheap. Wide crosses the network. Your job is slow because of a shuffle — assume that first, prove it in the UI.

5 · Storage $\neq$ compute

S3 holds the data forever and cheaply. Compute appears, does the work, and disappears. That is modern AWS data engineering.

6 · Format beats tuning

Parquet + sane partitioning will beat weeks of config tuning, every time. Fix the data layout before you touch a Spark conf.

If you walk out with only these six, you can hold your own in any data engineering design review. The rest is detail you will pick up by doing.

Where you go from here

The labs

- ▶ **01** — Spin up EMR Serverless, run your first job against S3
- ▶ **02** — Convert 50 GB of JSON to partitioned Parquet. Measure both.
- ▶ **03** — Build the funnel. Read `.explain()` and find the pushdown.
- ▶ **04** — Break it on purpose: skew it, then fix it in the Spark UI.
- ▶ **05** — Iceberg table, MERGE, time travel, then query it in Athena.

What to read next

- ▶ *Spark: The Definitive Guide* — chapters 1-5, then 15-19
- ▶ *Learning Spark, 2nd ed.* (free from Databricks)
- ▶ The Spark SQL performance tuning docs — short and worth it
- ▶ AWS: EMR Best Practices Guide, and the Glue developer guide
- ▶ The original Google MapReduce paper — 13 pages, still the clearest thing written

Habits that make you good fast

- ▶ `.explain()` before every run at scale
- ▶ Open the Spark UI **every single time**
- ▶ Know your partition count and your core count
- ▶ Search the built-in functions before writing a UDF
- ▶ Look at the bill weekly — cost is a performance metric

You are not learning a new way to think.

You already know map, filter and reduce. Everything today was about making those three survive 4 terabytes, forty machines, and hardware that dies while you watch. **Now go break something in lab 1.**

QUESTIONS

Ask the ones you think are **too basic**.

Those are the ones everyone else in the room is also stuck on.